



```
In [13]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import squarify

data = pd.read_csv("upi_fraud_file.csv");
data
```

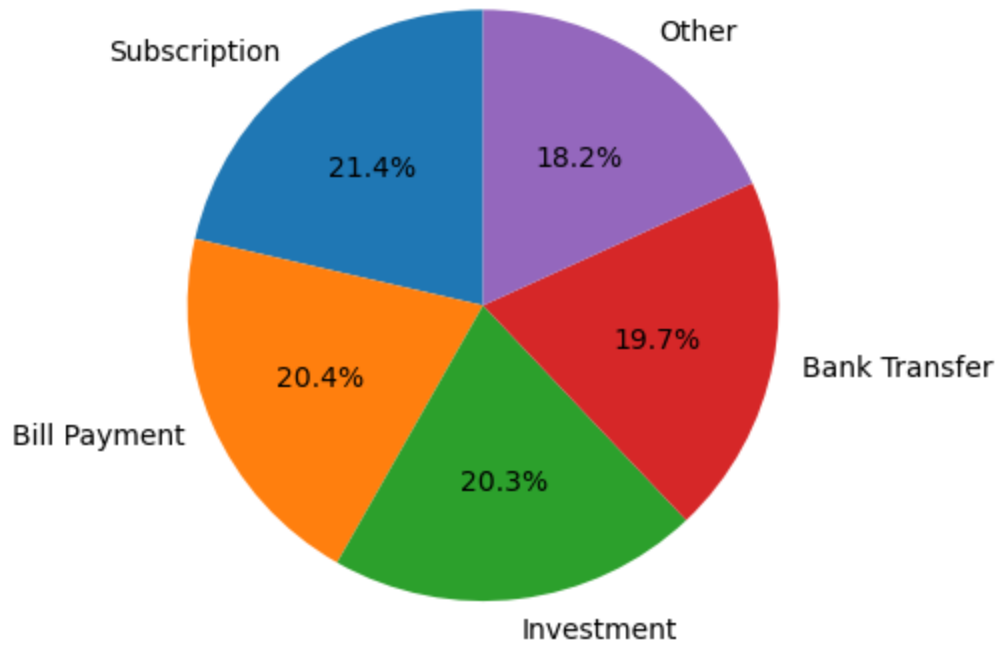
```
Out[13]:
```

	Transaction_ID	Date	Time	Merchant_ID
0	T00022452	20-04-2023	4:50:22 AM	f65a902b-2396-40cc-9593-97e103f1bc15
1	T00032233	8/8/2023	8:09:21 AM	d4a5efcb-4eb6-4d3a-8132-07bb3e6e13a4
2	T00037364	25-06-2023	7:49:01 AM	759ad138-9473-4729-8699-3d72c7ffb983
3	T00015924	28-01-2024	5:44:12 AM	d8f561e4-bded-4ef0-bcd8-5494b2e31a94
4	T00021805	21-06-2023	9:40:59 AM	26fdd7a1-8537-4dfe-bcf7-f5a127b36682
...	...	...	...	...
6995	T00015738	1/9/2023	4:42:08 AM	66a49583-d761-4907-bc8b-040baa803fef
6996	T00034532	12/7/2023	2:51:31 AM	c54bf108-583d-4166-bf0d-e7d426bd9b22
6997	T00012841	26-12-2023	11:12:34 AM	3e7c4c4f-3f6a-45f5-8701-9db5ec9d69bb
6998	T00040507	29-04-2023	10:34:38 AM	22198b06-ac35-43d8-8c46-4ee2cf9b3145
6999	T00047503	16-11-2023	5:42:35 AM	3712ead2-ae68-499a-a4b9-d702bb1ae750

7000 rows × 20 columns

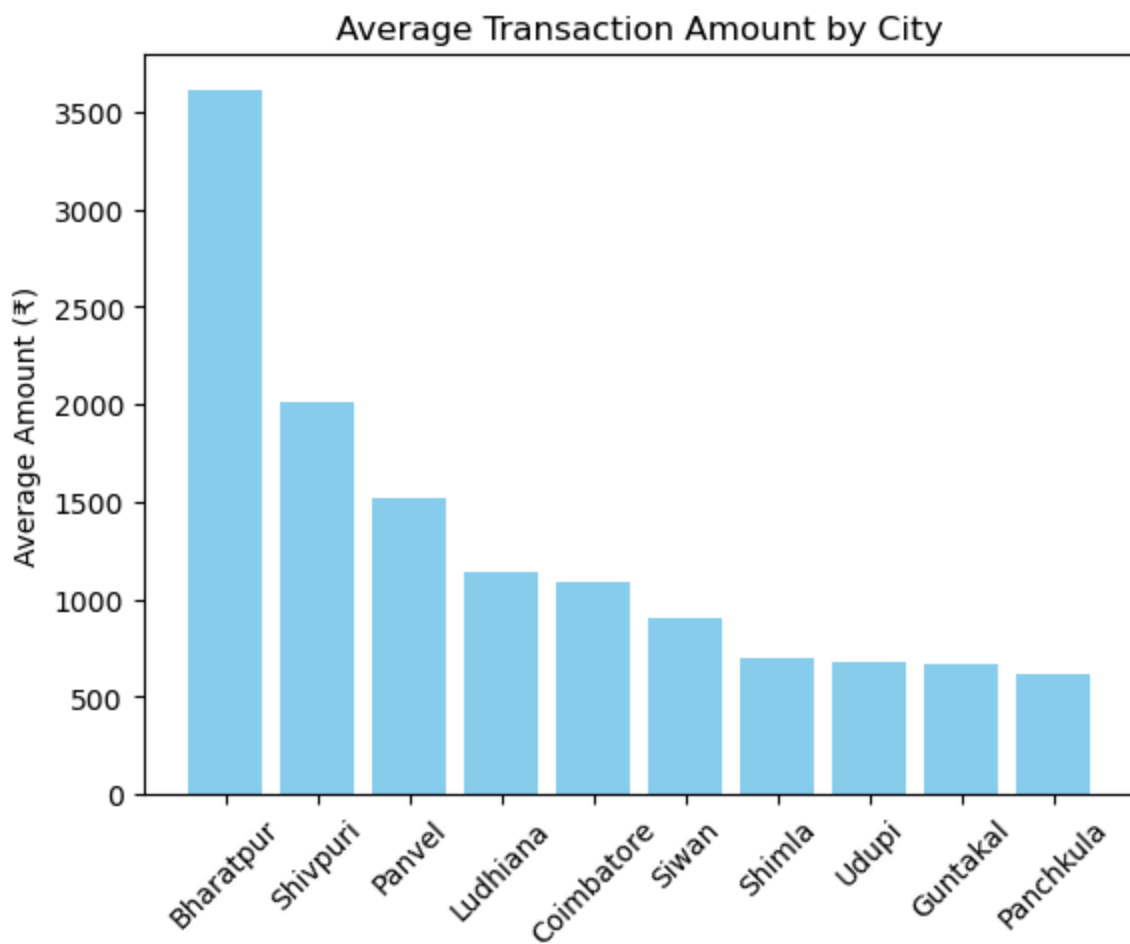
```
In [5]: #1.This graph shows how different kinds of transactions such as Refund, Bank T
type_count = data["Transaction_Type"].value_counts().head(5)
plt.pie(type_count, labels=type_count.index, autopct="%1.1f%%", startangle=90)
plt.title("Pie Chart - Transaction Type Distribution")
plt.show()
```

Pie Chart - Transaction Type Distribution

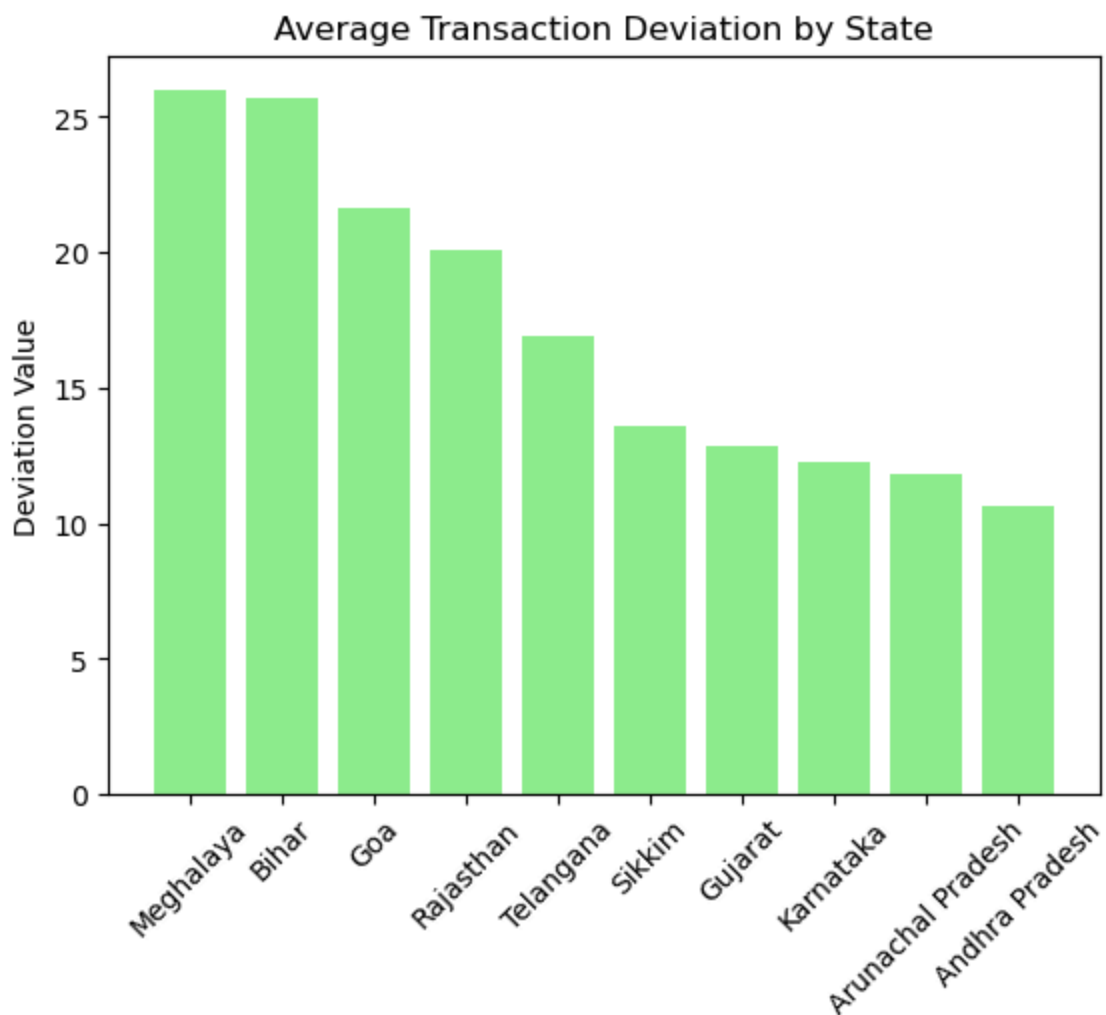


In []:

```
In [7]: #3.This chart compares the average transaction amount across top cities. It sh
city_avg = data.groupby("Transaction_City")["amount"].mean().nlargest(10)
plt.bar(city_avg.index, city_avg.values, color="skyblue")
plt.xticks(rotation=45)
plt.title("Average Transaction Amount by City")
plt.ylabel("Average Amount (₹)")
plt.show()
```



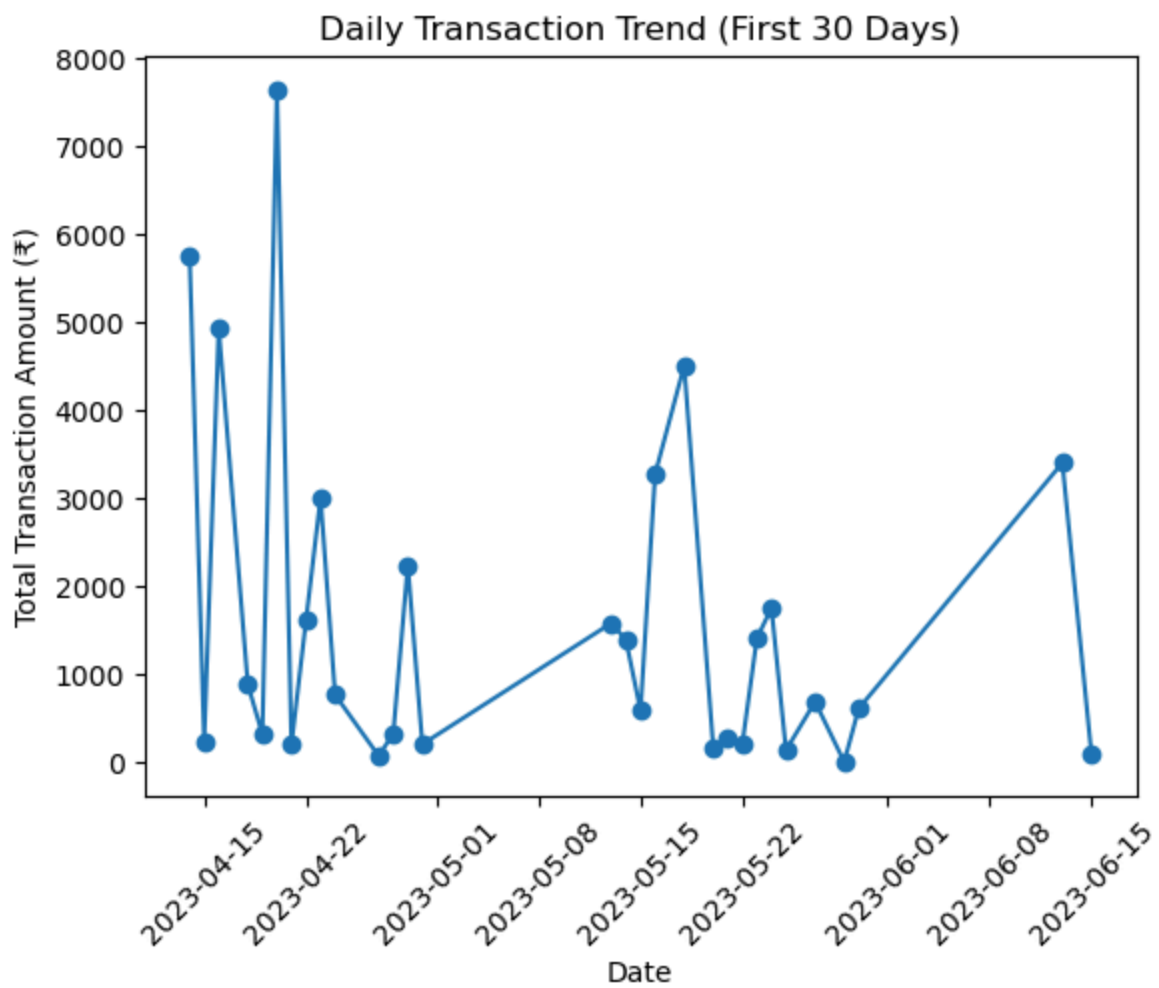
```
In [8]: #4.Shows how transaction deviation amounts vary from the average across states
state_dev = data.groupby("Transaction_State")["Transaction_Amount_Deviation"].
plt.bar(state_dev.index, state_dev.values, color="lightgreen")
plt.xticks(rotation=45)
plt.title("Average Transaction Deviation by State")
plt.ylabel("Deviation Value")
plt.show()
```



```
In [15]: #5.Displays the trend of transaction amounts over time, highlighting peaks or
data["Date"] = pd.to_datetime(data["Date"], dayfirst=True, errors="coerce")

daily_amount = (
    data.groupby("Date")["amount"]
        .sum()
        .sort_index()
        .head(30)
)

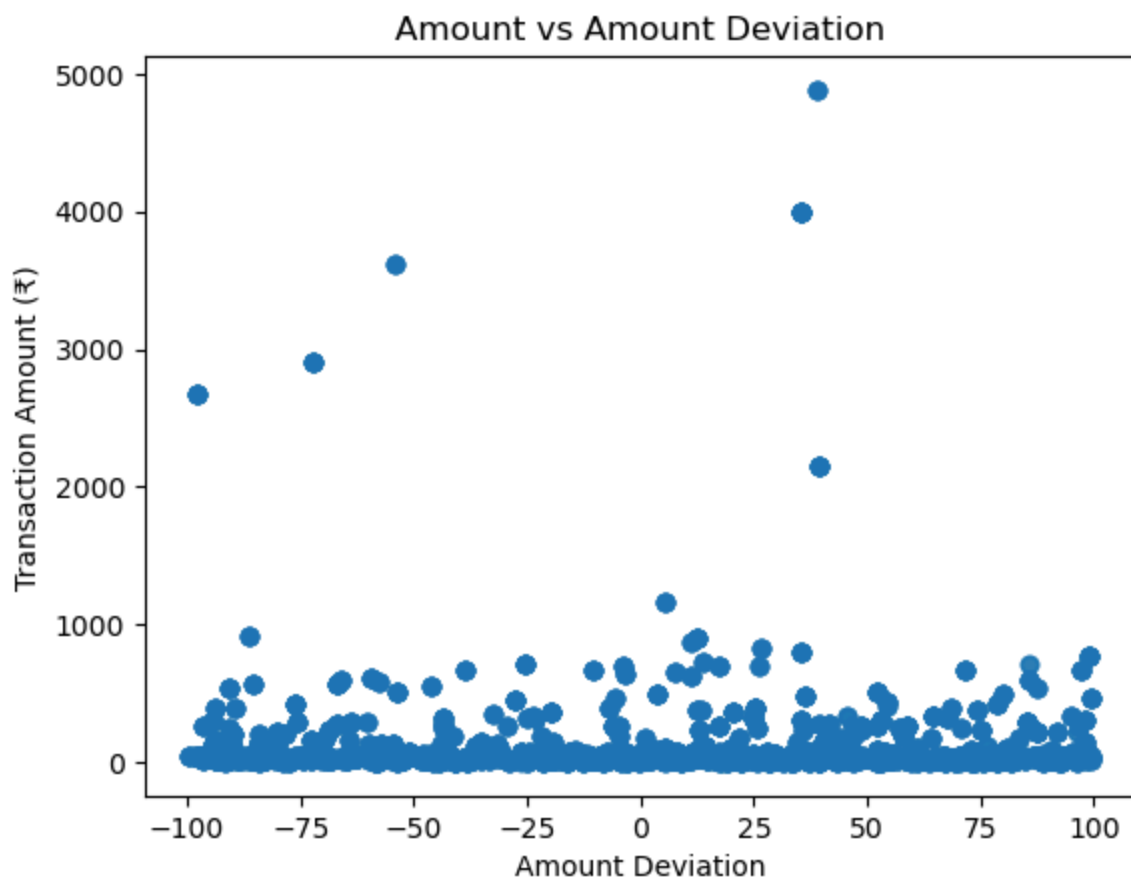
plt.plot(daily_amount.index, daily_amount.values, marker='o')
plt.title("Daily Transaction Trend (First 30 Days)")
plt.xlabel("Date")
plt.ylabel("Total Transaction Amount (₹)")
plt.xticks(rotation=45)
plt.show()
```



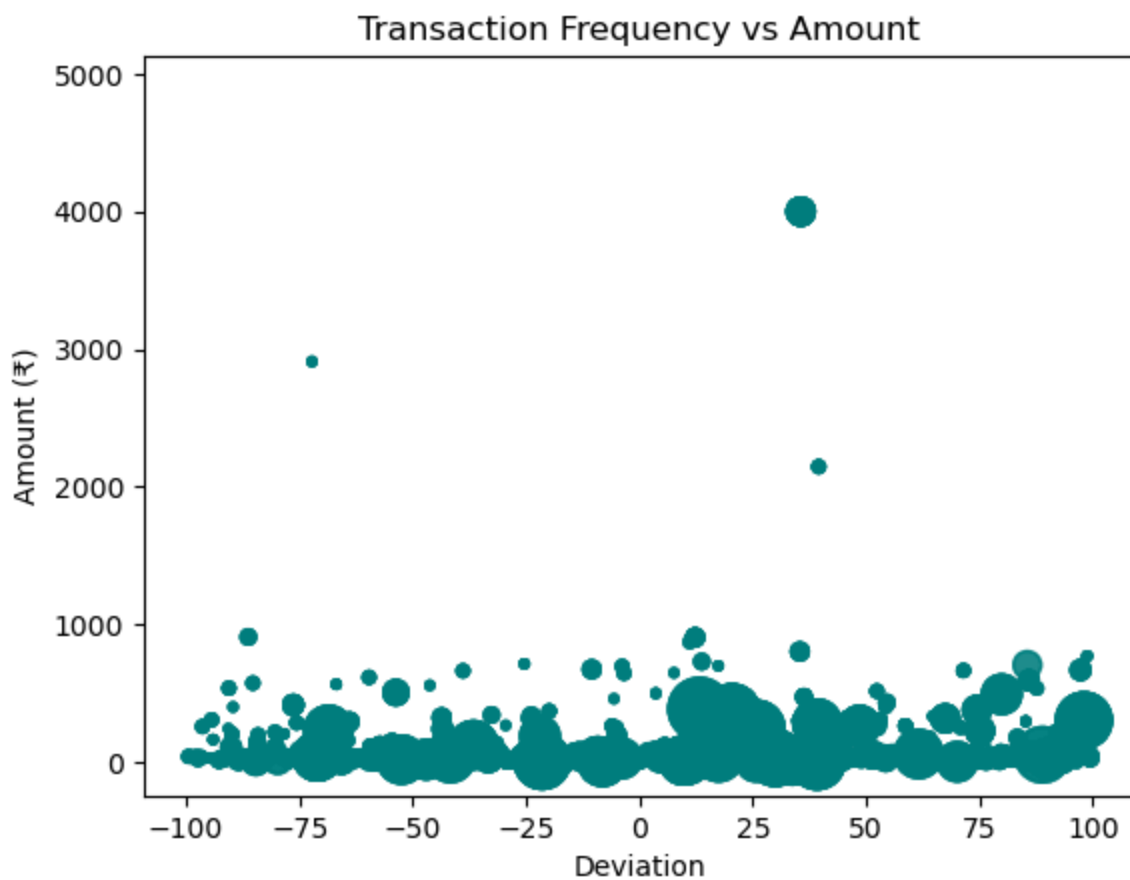
```
In [16]: #6.Represents total transaction volume over time, where the filled area emphasizes
plt.fill_between(np.arange(50), data["amount"].head(50), color="lightcoral", alpha=0.5)
plt.title("Transaction Volume Trend (First 50 Records)")
plt.xlabel("Transaction Index")
plt.ylabel("Amount (₹)")
plt.show()
```



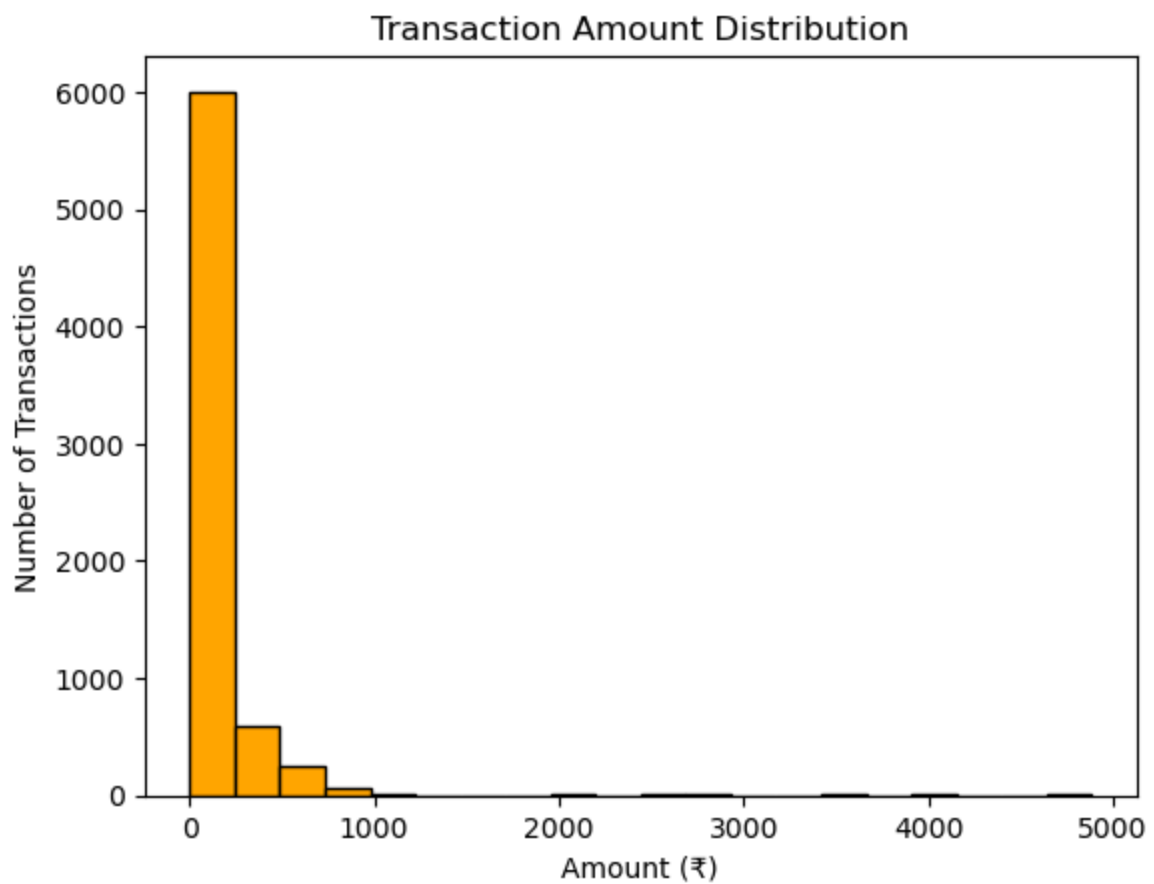
```
In [20]: #7.This plot relates Transaction Amount and Amount Deviation, revealing if lar
plt.scatter(data["Transaction_Amount_Deviation"], data["amount"], alpha=0.6)
plt.title("Amount vs Amount Deviation")
plt.xlabel("Amount Deviation")
plt.ylabel("Transaction Amount (₹)")
plt.show()
```



```
In [21]: #8.Displays how the number of transactions affects spending. Bubble size repre
plt.scatter(
    data["Transaction_Amount_Deviation"],
    data["amount"],
    s=data["Transaction_Frequency"]*10,
    alpha=0.5,
    color="teal"
)
plt.title("Transaction Frequency vs Amount")
plt.xlabel("Deviation")
plt.ylabel("Amount (₹)")
plt.show()
```

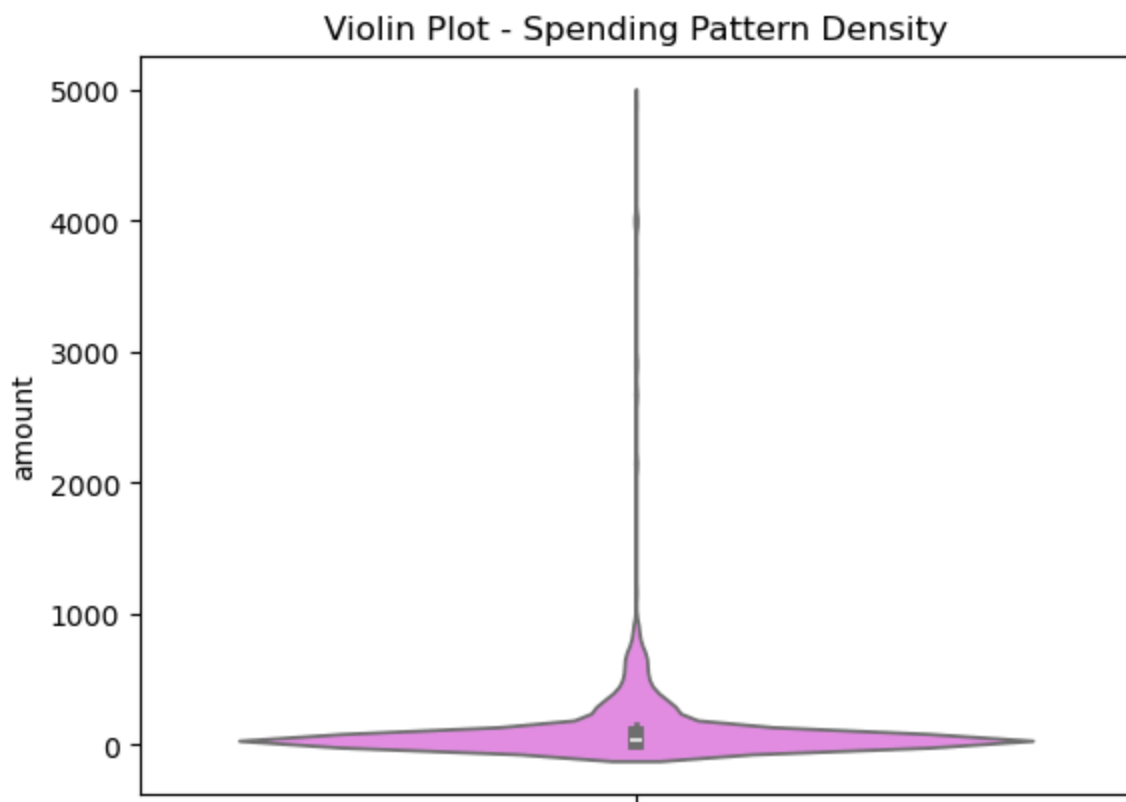


```
In [22]: #9.Shows how transaction amounts are spread – whether most transactions are low
plt.hist(data["amount"], bins=20, edgecolor="black", color="orange")
plt.title("Transaction Amount Distribution")
plt.xlabel("Amount (₹)")
plt.ylabel("Number of Transactions")
plt.show()
```

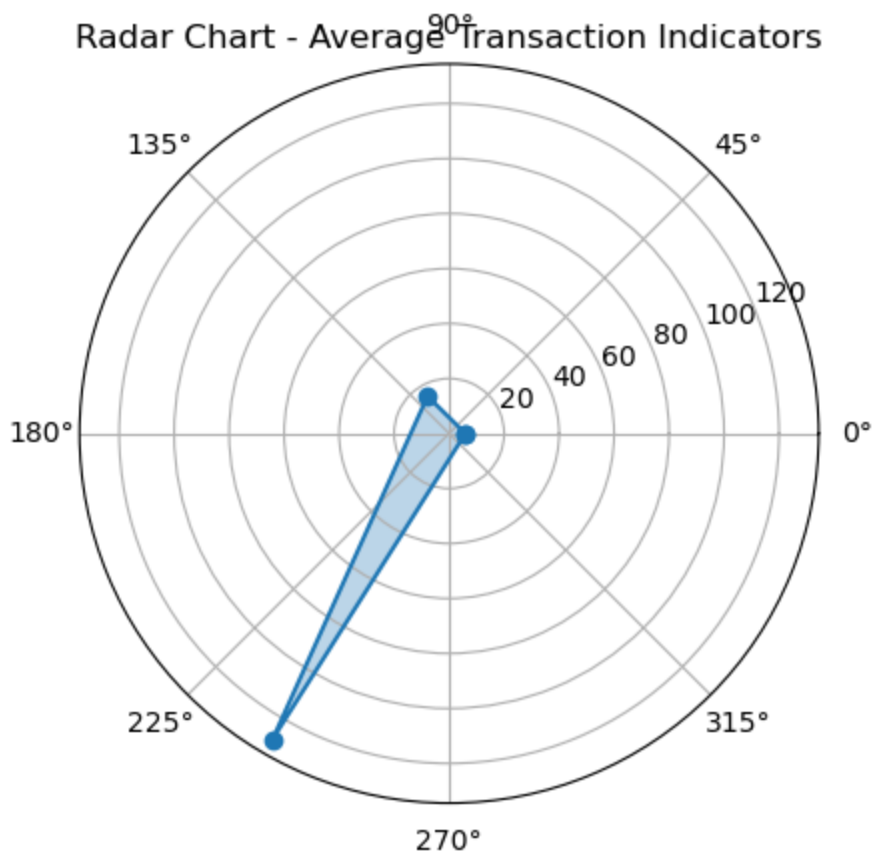



In []:

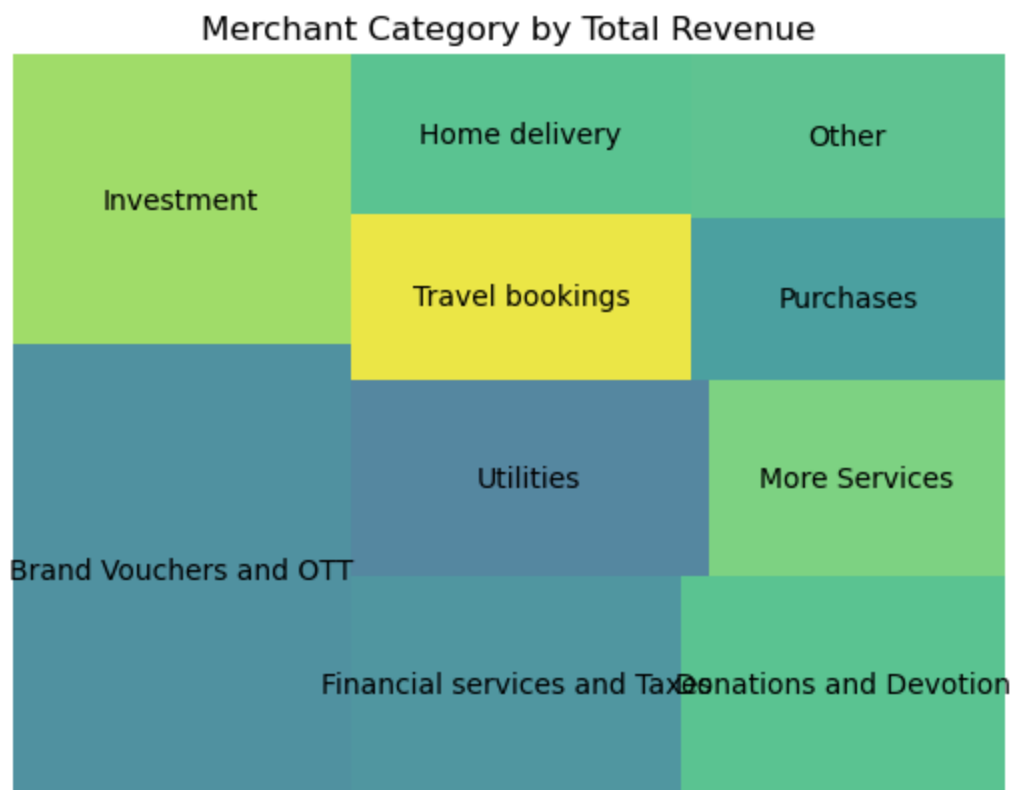
```
In [24]: #11.Shows how densely transaction amounts are concentrated within different va
sns.violinplot(y=data["amount"], color="violet")
plt.title("Violin Plot - Spending Pattern Density")
plt.show()
```



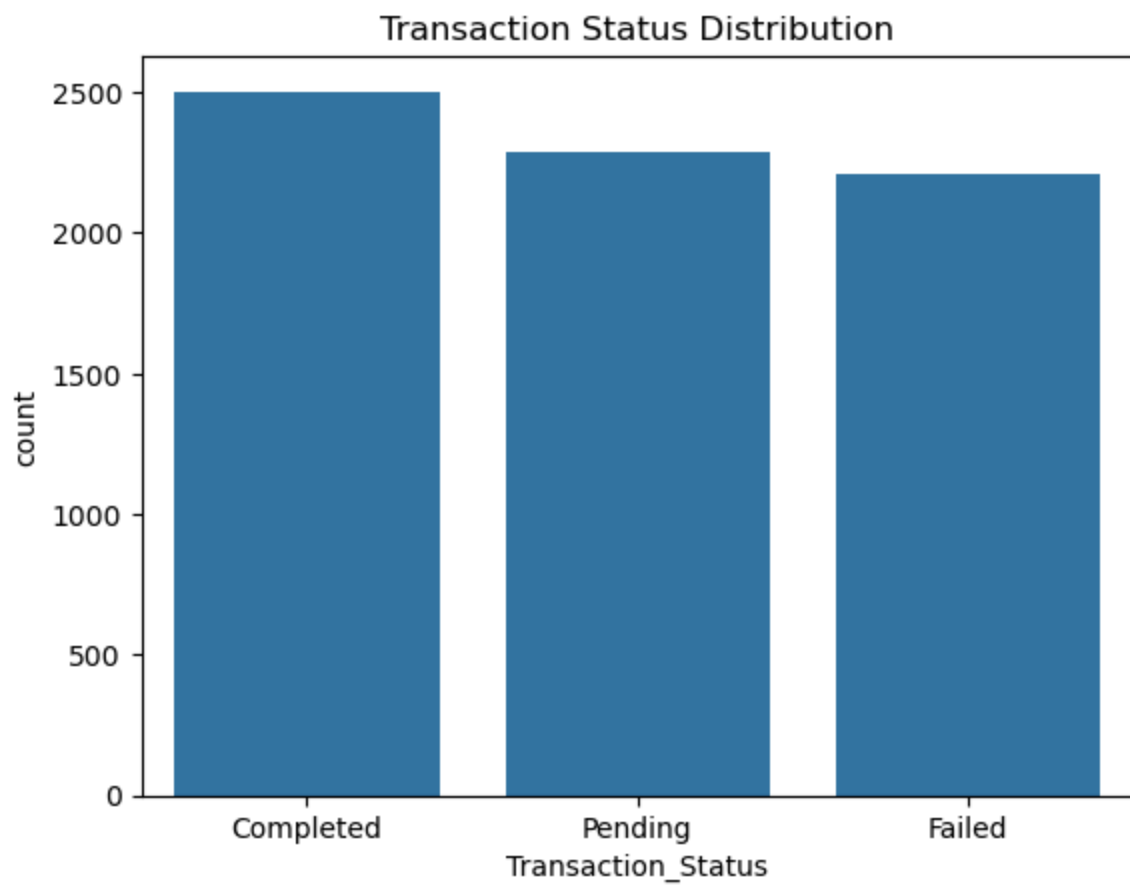
```
In [25]: #12.Compares average values of deviation, amount, and days since last transact
cols = ["Transaction_Amount_Deviation", "Days_Since_Last_Transaction", "amount"]
vals = [data[c].mean() for c in cols]
vals += vals[:1]
angles = np.linspace(0, 2*np.pi, len(cols), endpoint=False).tolist() + [0]
plt.polar(angles, vals, marker='o')
plt.fill(angles, vals, alpha=0.3)
plt.title("Radar Chart - Average Transaction Indicators")
plt.show()
```



```
In [26]: #13.Illustrates total transaction amounts contributed by each merchant category
cat_sum = data.groupby("Merchant_Category")["amount"].sum().nlargest(10)
squarify.plot(sizes=cat_sum.values, label=cat_sum.index, alpha=0.8)
plt.title("Merchant Category by Total Revenue")
plt.axis("off")
plt.show()
```



In [27]: *#14. Displays the count of transactions by their status: Completed, Pending, or*
`sns.countplot(x="Transaction_Status", data=data)`
`plt.title("Transaction Status Distribution")`
`plt.show()`



In []: